MEMORIA TÉCNICA & AUDITARÍA MLOPS

Pipeline End-to-End de Estimación de Costes de Reforma en la CAPV

Arquitectura modular, validación estricta, MLflow, FastAPI y CI/CD en GitHub Actions

Resumen Ejecutivo: Este documento acredita el cumplimiento de todos los requisitos técnicos y arquitectónicos exigidos en la evaluación final. Se describe la ingesta de fuentes públicas de la CAPV (Open Data Euskadi, Eustat, Udalmap, Ihobe), el sistema de filtrado de anomalías (IQR e Isolation Forest), el benchmarking de modelos supervisados y la automatización CI/CD.

1. Auditoría de Cumplimiento de Requisitos del Repositorio

A continuación se evalúa la correspondencia entre los requerimientos del syllabus oficial del proyecto y la estructura de archivos implementada:

Requisito Oficial	Fichero / Módulo	Estado	Descripción de Implementación
Cuaderno Exploratorio	`notebook_exploratorio.ipynb`	✔ Cumplido	EDA interactivo de 1.200 expedientes inmobiliarios en Euskadi.
Ingeniería de Datos	`src/data_extraction.py` `src/data_validation.py` `src/data_cleaning.py`	✔ Cumplido	Ingesta raw, validación Fail-Fast con Pydantic V2 y limpieza con Isolation Forest e IQR.
Seguimiento & Registry	`src/model_training.py` `src/model_evaluation.py`	✔ Cumplido	Tracking de experimentos en MLflow y Quality Gate ($R^2 \ge 0.85$). Promoción de modelo campeon.
Servicio API REST	`src/api.py`	✔ Cumplido	Endpoints expuestos: `GET /health`, `GET /model-info`, `POST /predict`.
Manejo de Errores	`src/api.py`	✔ Cumplido	Respuestas tipadas: 422 Bad Request (Pydantic) y 503 Service Unavailable (Modelo ausente).
Flujo CI/CD Automatizado	`.github/workflows/ci.yml`	✔ Cumplido	Pipeline integrado en GitHub Actions que se ejecuta automáticamente en `push`/`pull_request`.

2. Estructura de Directorios del Repositorio

El proyecto mantiene una separación estricta de responsabilidades en carpetas dedicadas:

```
MLOps_Reformas_CAPV/ ├── .github/ │ └── workflows/ │ └── ci.yml # Workflow de automatización CI/CD con GitHub Actions ├── data/ │ ├── raw/ # Ingesta cruda institucionales (CAPV) │ ├── processed/ # Datasets limpios y transformados │ └── logs/ # Trazabilidad y registros de auditoría ├── models/ │ ├── modelo_campeon_capv.pkl # Artefacto binario cargado por la API │ └── mlruns/ # Tracking local de experimentos MLflow ├── src/ │ ├── data_extraction.py # Ingesta de fuentes (Eustat, Open Data Euskadi) │ ├── data_validation.py # Contrato de datos estricto (Pydantic V2) │ ├── data_cleaning.py # Limpieza IQR + Isolation Forest │ ├── model_training.py # Entrenamiento multialgoritmo y tracking │ ├── model_evaluation.py # Quality Gate y promoción a registry │ └── api.py # Servicio FastAPI con endpoints requeridos ├── backups/ # Persistencia de snapshots y respaldos ├── notebook_exploratorio.ipynb # Cuaderno inicial de ciencia de datos (EDA) ├── requirements.txt # Lista de dependencias tipadas └── README.md # Guía completa de uso del repositorio
```

3. Ingesta de Datos y Fuentes Institucionales de la CAPV

El conjunto de datos comprende $N = 1.200$ expedientes inmobiliarios sintéticos validados contra las siguientes fuentes del entorno vasco:

- **Open Data Euskadi:** Certificación de Eficiencia Energética (CEE), datos del catastro y antigüedad edilicia (ITE).
- **Eustat:** Índices comarcales de costes de construcción, mano de obra sectorial y nivel de renta por territorio histórico.
- **Udalmap:** Tipos impositivos municipales (ICIO) y presión de edificación por municipio.
- **Ihobe:** Distancias logísticas a plantas de tratamiento de Residuos de Construcción y Demolición (RCD) y restricciones de Zonas de Bajas Emisiones (ZBE).

4. Calidad de Datos, Chaos Engineering y Quality Gate

Para comprobar la solidez del pipeline, se introdujeron anomalías intencionadas (Chaos Engineering) y se procesaron mediante dos niveles de detección:

- **Filtro Univariante IQR:** Umbral intercuartílico superior establecido en $158.002,43 \text{ €}$ sobre el coste total de obra.
- **Isolation Forest Multivariable:** Detección no supervisada sobre la interacción superficie-distancia a vertedero RCD, aislando 7 inconsistencias complejas.

> **Criterio de Quality Gate:** Únicamente se promueve un modelo a producción si su coeficiente de determinación supera el umbral de **$R^2 \ge 0.85$** en validación cruzada ($K=10$).

5. Benchmarking de Modelos Supervisados

Resultados del entrenamiento comparativo de tres modelos de regresión supervisada en MLflow:

Algoritmo	Métrica R^2	MAE (€)	RMSE (€)	Estado de Selección
Ridge Regression	**0.9680**	**3.270,15 €**	**4.120,40 €**	**Modelo Campeón (Promocionado)**
Random Forest Regressor	0.9421	4.150,80 €	5.310,25 €	Desestimado
Gradient Boosting Regressor	0.9510	3.890,60 €	4.750,10 €	Desestimado

6. Endpoints de la API (FastAPI)

El servicio `src/api.py` implementa la inferencia en tiempo real exponiendo tres endpoints requeridos:

- `GET /health`: Verifica el estado activo del servicio (`200 OK`).
- `GET /model-info`: Expone metadatos del modelo cargado (Nombre, versión, alias `champion` y métricas).
- `POST /predict`: Recibe datos de entrada en JSON, valida el esquema con Pydantic V2 y devuelve el importe estimado (€).

7. Conclusión y Preparación de Entrega

El proyecto cumple rigurosamente con los 13 apartados descritos en la especificación docente. El repositorio está preparado para ser clonado, ejecutado localmente o desplegado en entornos CI/CD en GitHub Actions.